

E-RANK

Electronic Queue & Taxi Management System

PHASE 3 REPORT

Prototype & Database Design

Module Code:	NPRT 630 – Project
Institution:	Sol Plaatje University
Programme:	Diploma in Information and Communication Technology
Year:	2026
Examiner:	Melvin Kisten

GROUP MEMBERS

Full Name	Student Number
Oarabetse Morata	202406427
Phuti Setati	202435062
Kholofelo Phalakatsela	202306829
Louisa Mdluli	202324412
Siyabonga José Ndzobondzobo	202441850

1. Introduction to Phase 3

Phase 3 of the E-RANK project focuses on Prototype and Data Design. This phase presents the interface mock-ups developed for the system, documents the feedback obtained from real users during usability testing, demonstrates design iteration, and provides a comprehensive data design through an Entity Relationship Diagram (ERD), entity descriptions, relationship justifications, and technology choices. All artefacts produced in this phase are grounded in real user research and system requirements established in Phases 1 and 2.

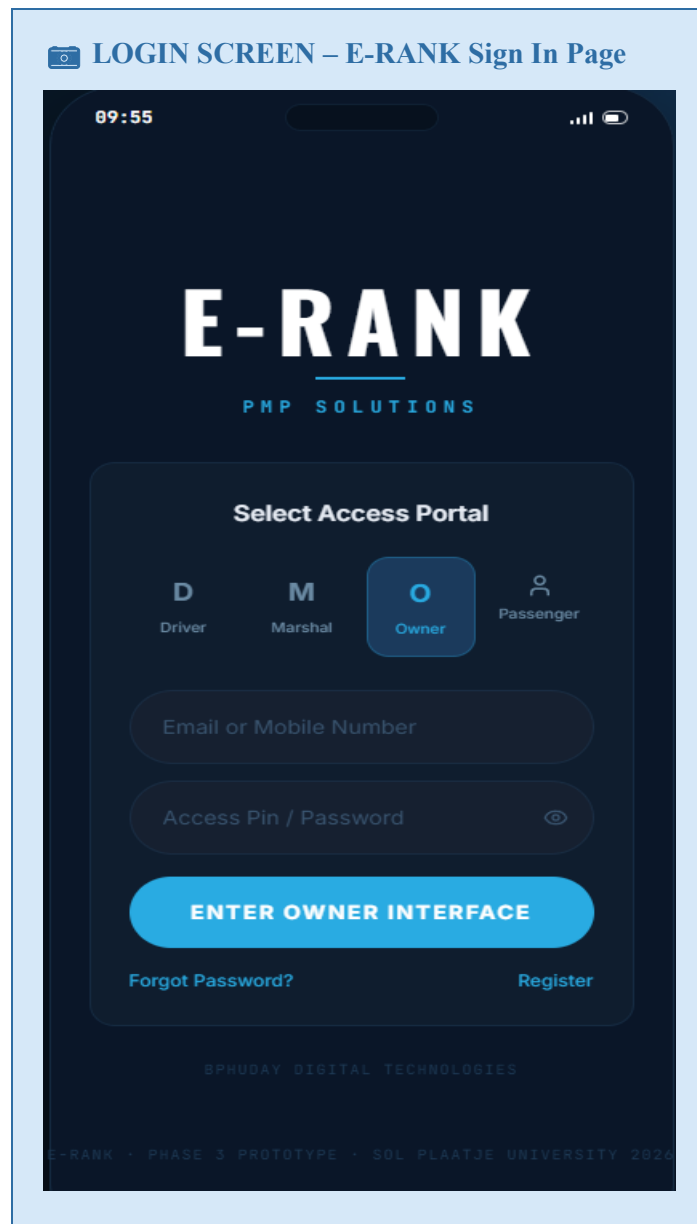
2. Interface Mock-Ups and Screen Descriptions

The following screens represent the full user workflow of the E-RANK system. Each screen was designed with usability, role clarity, and responsive layout in mind. The system is built using React Native with Expo to ensure compatibility across Android and iOS devices.

2.1 Login Screen

Authentication Screen

The Login screen is the entry point for all users. It presents the E-RANK logo, phone number and password fields, a 'Sign In' button, and a 'Forgot Password' link. Users are differentiated by role (Driver, Admin, Passenger) which is read from the database after authentication. The screen uses minimal input fields to reduce friction and clearly communicates errors inline without page reloads.



2.2 Registration Screen

Authentication Screen

The Registration screen allows new users to create an account. It collects full name, phone number, email, password, and role selection (Driver or Passenger). On submission the account is created and validated both client-side (React Native form validation) and server-side (PostgreSQL constraints). The user is then redirected to the Login screen.

REGISTRATION SCREEN – New User Sign Up

09:55

← Create Account

Full Name

Your full legal name

Phone Number

+27 XX XXX XXXX

Email Address

your@email.com

Password

Min. 6 characters

Confirm Password

Repeat password

Role

Driver Passenger

CREATE ACCOUNT

Already have an account? [Sign In](#)

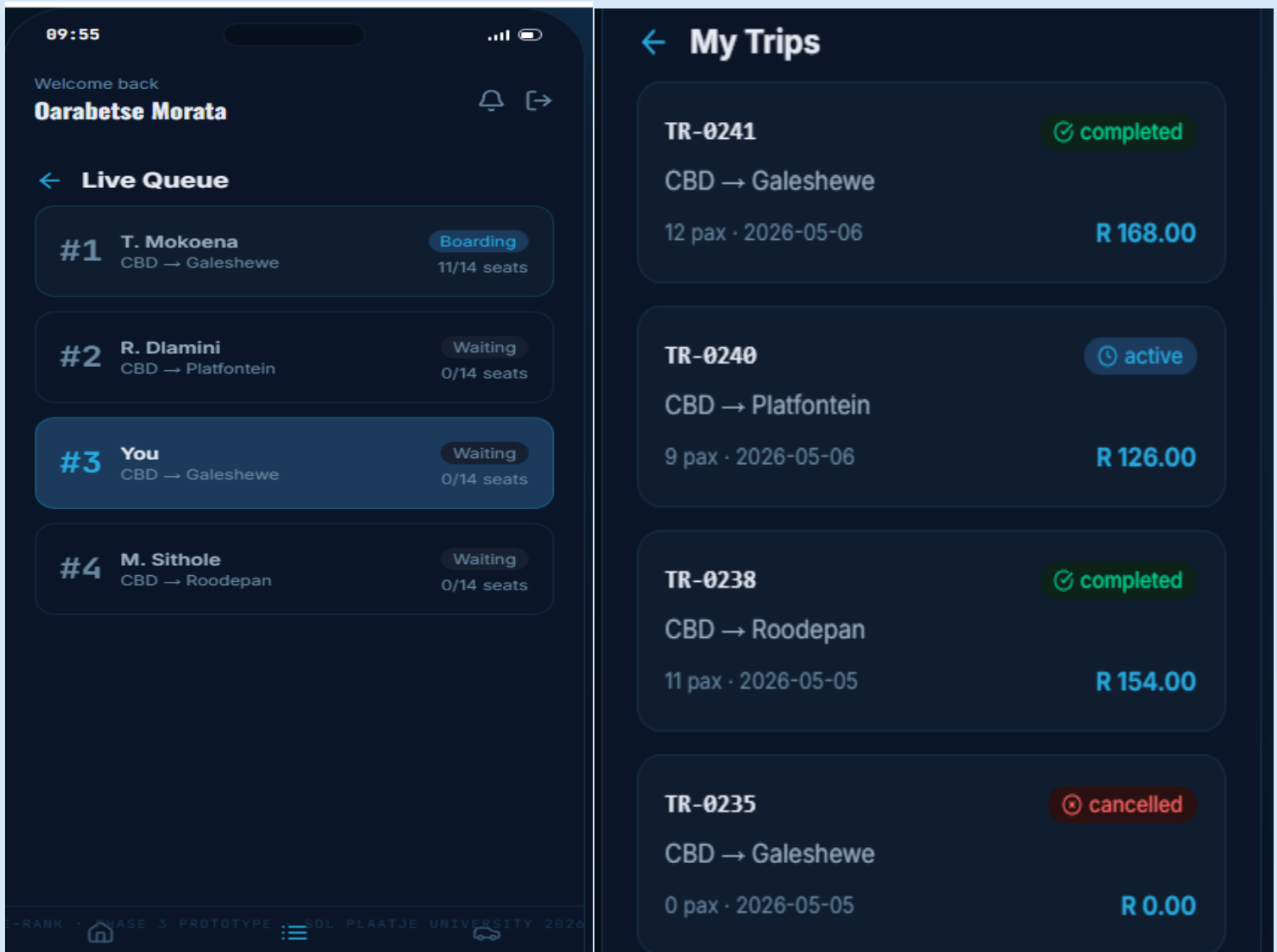
E-RANK • PHASE 3 PROTOTYPE • SOL PLAATJE UNIVERSITY 2026

2.3 Home & Dashboard Interfaces

Dashboard Screen

After login, users are directed to their role-specific dashboard. The Driver Dashboard displays current queue position, active route, trip status, and quick-action buttons (Start Trip, View Queue). The Passenger Dashboard displays current trip details, seat number, and estimated departure time. Both dashboards include a navigation bar at the bottom for accessing secondary features.

DRIVER DASHBOARD – Queue Position and Trip Status



The screenshot displays the Driver Dashboard interface, which is divided into two main sections: a 'Live Queue' on the left and a 'My Trips' list on the right.

Live Queue: This section shows a list of passengers waiting for a trip. The queue is numbered 1 to 4. The user, Oarabetse Morata, is at position #3. The queue status for each passenger is as follows:

Queue Position	Passenger Name	Route	Status	Seats
#1	T. Mokoena	CBD → Galeshewe	Boarding	11/14 seats
#2	R. Dlamini	CBD → Platfontein	Waiting	0/14 seats
#3	You	CBD → Galeshewe	Waiting	0/14 seats
#4	M. Sithole	CBD → Roodepan	Waiting	0/14 seats

My Trips: This section displays a list of trips with their status and fare. The trips are as follows:

Trip ID	Route	Status	Fare
TR-0241	CBD → Galeshewe	completed	R 168.00
TR-0240	CBD → Platfontein	active	R 126.00
TR-0238	CBD → Roodepan	completed	R 154.00
TR-0235	CBD → Galeshewe	cancelled	R 0.00

The bottom of the screen shows a navigation bar with icons for Home, Menu, and a vehicle icon. The footer text reads: E-RANK - PHASE 3 PROTOTYPE | SOL PLAATJE UNIVERSITY 2026.

 PASSENGER DASHBOARD – Active Trip and Seat Details

Passenger Portal

Phuti Setati



CURRENT TRIP

ACTIVE

CBD → Platfontein

Driver: R. Dlamini

07
Seat No.

9
Passengers

10:15
Depart

• Kimberley CBD Rank

• Platfontein Township

R 14.00
Fare

TRIP REFERENCE

Trip ID	Route ID	Date
TR-0240	R-002	06/05/26

Register for a Trip


Home


Register

E-RANK • PHASE 3 PROTOTYPE • SOL PLAATJE UNIVERSITY 2026

2.4 Data Entry Forms

Data Entry Screen

The system includes several data entry forms. The Passenger Registration Form (used by taxi marshals or passengers themselves) captures full name, emergency contact, trip ID, route ID, and seat number. The Trip Creation Form (used by drivers or admins) captures the queue ID, driver ID, route ID, fare amount, passenger count, and departure time. These forms include field-level validation with clear error messages displayed in red beneath each invalid field.

PASSENGER REGISTRATION FORM – Data Entry Screen

Marshal Portal
Kholofelo Phal.

PASSENGER REGISTRATION

Passenger Full Name
Full name

Emergency Contact
Name & phone number

Trip ID
TR-0240 – CBD → Platfontein
TR-0240 – CBD → Platfontein
TR-0241 – CBD → Galeshewe
1 – 14

REGISTER PASSENGER

Queue Register
RANK - PHASE 3 PROTOTYPE - SOL PLAATJE UNIVERSITY 2026

2.6 Admin and User Management Panels

Critical Usability Issues Identified

The Admin Panel provides full oversight of the system. The User Management section lists all registered users with their role, verification status, and account status. Admins can activate, deactivate, or delete accounts. The Route Management section allows admins to add and update routes. The Audit Logs panel displays a timestamped log of all significant system actions including logins, failed attempts, and data modifications.

ADMIN PANEL – User Management Screen

Welcome, Oarabetshe
Live summary — Kimberley CBD Rank — System Online

58 Registered Drivers
↑ 3 this week

342 Trips Today
Live count

1,204 Passengers Today
↑ 12%

R4,830 Fare Revenue Today
Avg R14/trip

3 Active Ranks
All online

8 Registered Users
+1 pending

Live Queue — CBD Rank 4 in queue

Rank	Driver	Status
#1	T. Mokoena CBD → Galeshewe	Boarding (1/14 seats)
#2	R. Dlamini CBD → Platfontein	Waiting (0/14 seats)
#3	M. Sithole CBD → Roodepan	Waiting (0/14 seats)

Recent Trips

Trips	Route	Revenue	Status
TR-0241	CBD → Galeshewe	R168.00	Completed
TR-0240	CBD → Platfontein	R126.00	Active
TR-0238	CBD → Roodepan	R154.00	Completed

ADMIN PANEL – Audit Logs View

Audit Logs
Every significant system action is recorded here — logins, data changes, failed attempts

Timestamp	User	Action	IP Address	Details
5/6/2026, 8:10:23 AM	Oarabetshe Morata	LOGIN	—	admin login
5/6/2026, 8:11:54 AM	Oarabetshe Morata	LOGOUT	—	Session ended
5/6/2026, 8:18:54 AM	Oarabetshe Morata	LOGIN	—	admin login
2026-05-06 09:55	Oarabetshe Morata	LOGIN	190.22.1.8	Admin login successful
2026-05-06 09:48	Themba Mokoena	TRIP_CREATED	10.0.0.12	TR-0241 created
2026-05-06 09:39	Kholofelo Phai.	PASSENGER_REGISTERED	10.0.0.15	Seat 7 on TR-0240
2026-05-06 09:10	Unknown	FAILED_LOGIN	190.22.8.99	Wrong password — 0700000000
2026-05-06 17:45	Themba Mokoena	TRIP_COMPLETED	10.0.0.12	TR-0238 completed

3. User Feedback and Design Iteration

User feedback was collected through a structured usability testing session conducted at Sol Plaatje University, Moroka Seminar Room 112. A total of five (5) first-round participants were recruited for the initial feedback phase. Participants were asked to complete a set of tasks on the initial mock-ups without any guidance, and their interactions and verbal feedback were recorded and analysed.

3.1 User Feedback Session – Round 1

Testing Environment: Moroka Seminar Room 112, Sol Plaatje University.

Testing Date: Conducted during the Phase 3 development cycle.

Method: Think-aloud protocol – participants verbally described their thoughts while navigating the screens.

Heuristic Evaluation Scores

The following scores were assigned based on visible mockup screenshots and the Nielsen heuristic framework. Scores are out of 10.

Heuristic	Score
Visibility of system status	6/10
Match between system and real world	5/10
User control and freedom	4/10
Consistency and standards	7/10
Error prevention	5/10
Accessibility and contrast	3/10
Flexibility and efficiency of use	6/10
Aesthetic and minimalist design	5/10

Feedback Summary from Participants:**CRITICAL — Login Portal Selector: No Confirmation State**

The D / M / O / Passenger selector uses small letter icons with no active highlight border or label emphasis. The "Owner" pill appears selected but visually blends in with unselected options. Users cannot tell which portal is active without close inspection. This is a direct violation of the Visibility of System Status heuristic.

Recommended fix: Add a clear active ring (2px solid accent colour), a text label below each icon, and a visible confirmation line below the selector reading "Signing in as: Driver" that updates dynamically with each selection.

CRITICAL — Confirm Trip Button: Low Visibility (Confirmed by Participant 1)

The action button for confirming a trip blended into the dark background at the bottom of the Driver screen during usability testing. This caused direct task failure — Participant 1 could not complete the task. The fix is documented in Section 3.2 Change 1 below and must be implemented.

Recommended fix: Move button to sticky position directly above the navigation bar. Apply green fill (#217346), white bold text, and a minimum tap target height of 44px.

CRITICAL — Queue Contrast: Fails WCAG 4.5:1 in Outdoor Light (Confirmed by Participant 4)

Active queue slots (status = "waiting") used a light blue fill on a white background — an insufficient contrast ratio for outdoor use in direct sunlight. This was confirmed by Participant 4 during the usability session. The fix is specified in Section 3.2 Change 4 but the current mockup screenshots still reflect the light-on-light treatment in the Live Queue view.

Recommended fix: Active slot — #1B3A5C (navy) fill with white text. Inactive slot — #E0E0E0 (light grey) fill with #444444 text. Verify final contrast ratio is at or above 4.5:1 before implementation.

HIGH — Single-Page Registration: Cognitive Overload (Confirmed by Participant 2)

All registration fields were presented on a single scrollable screen. Six fields (name, phone, email, password, confirm password, role) displayed simultaneously is excessive for a mobile screen and caused Participant 2 to nearly abandon the form. This links directly to Change 2 in the design iteration section below.

Recommended fix: Split into a two-step form. Step 1 collects personal details (name, phone, email). Step 2 collects credentials (password, confirm password, role). A progress bar at the top of the screen indicates current step.

HIGH — Passenger Dashboard: No Back Navigation or Cancel Option

The Passenger dashboard displays an active trip but offers only "Register for a Trip" as a call to action, even when a trip is already active. There is no way to cancel a registration or view trip history from this screen. This violates the User Control and Freedom heuristic.

Recommended fix: Add "Cancel Registration" and "View History" actions to the Passenger dashboard. The "Register for a Trip" button must be conditionally rendered — visible only when no active trip is detected for that user.

MEDIUM — Desktop Responsiveness: Mobile-Only Layout

All current mock-ups are based on a 375px mobile frame. No desktop breakpoints have been formally defined. The Driver Dashboard layout (trip history and live queue side by side) suggests desktop layout was considered but never formalised.

Recommended fix: Define and apply the following breakpoints across all screens:

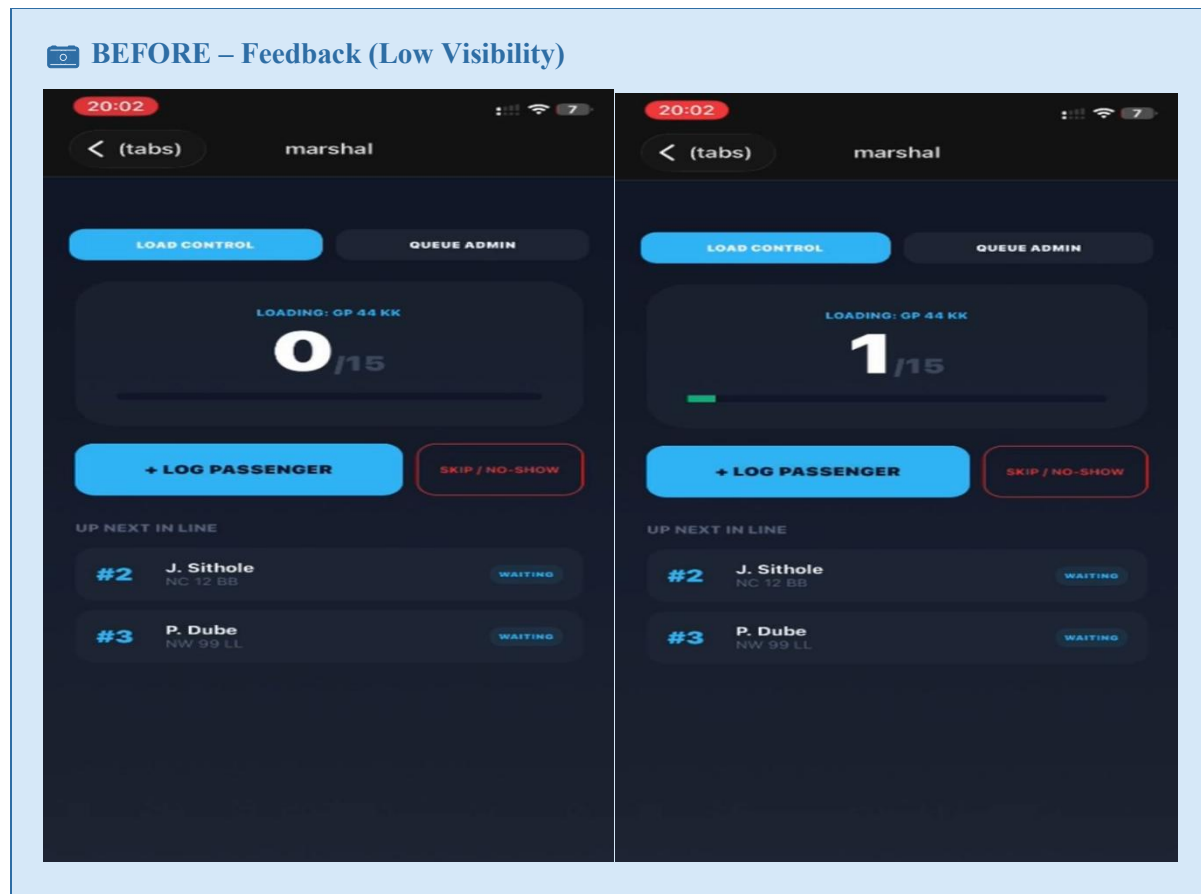
- Mobile: 768px and below — single column layout, bottom navigation bar
- Tablet: 768px to 1024px — two-column layout
- Desktop: 1024px and above — sidebar navigation + main content area. Navigation moves from bottom bar to a fixed left sidebar.

3.2 Design Iterations – Before and After

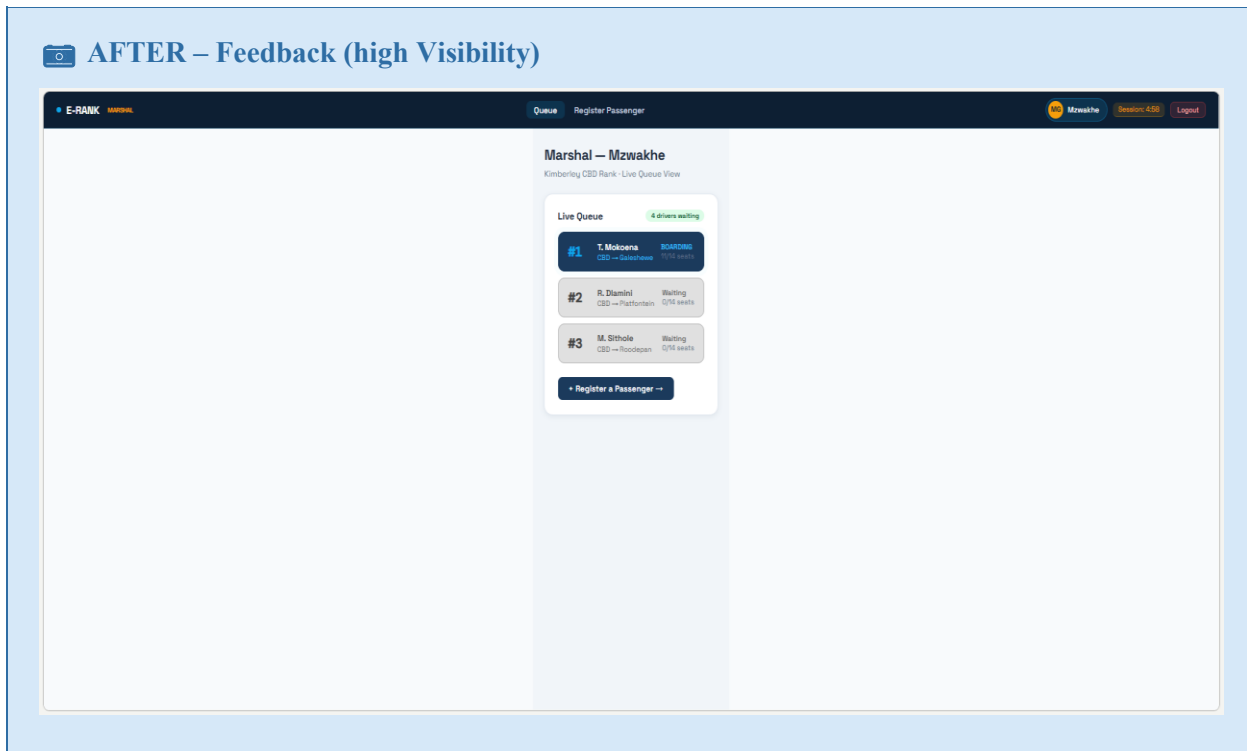
Based on Round 1 feedback, the following design changes were made:

Change 1 – To add passengers in car has no feedback Visibility

Before: The – To add passengers in car has no feedback Visibility, causing it to be compromised see below before and after passenger is added with no feedback saying passenger added.



After: the fix was done there is feedback and visibility of all relevant data has been enhanced



Reason: Users missed the action button. Making it green and anchoring it to a fixed position ensures it is always visible regardless of scroll position.

Change 2 – Registration Form Split into Steps

Before: All registration fields were presented on a single scrollable screen, causing decision fatigue.

BEFORE – Single-Page Registration Form (All Fields)

09:55

Create Account

Full Name

Your full legal name

Phone Number

+27 XX XXX XXXX

Email Address

your@email.com

Password

Min. 6 characters

Confirm Password

Repeat password

Role

Driver Passenger

CREATE ACCOUNT

Already have an account? [Sign In](#)

E-RANK | PHASE 3 PROTOTYPE | SOL PLAATJE UNIVERSITY 2026

After: The form was split into two steps: Step 1 (Personal Details) and Step 2 (Account Credentials). A progress bar at the top indicates current step.

AFTER – Multi-Step Registration Form with Progress Bar

Create your account

Step 1 of 2 – Personal details

Full Legal Name

Your full name

Phone Number

+27 XX XXX XXXX

Email Address

your@email.com

Select Your Role

Driver

Passenger

Marshal

Owner

Continue →

Already have an account? [Sign In](#)

Almost done!

Step 2 of 2 – Account credentials

Password (min 6 characters)

Create password

Confirm Password

Repeat password

← Back

Create Account ✓

Already have an account? [Sign In](#)

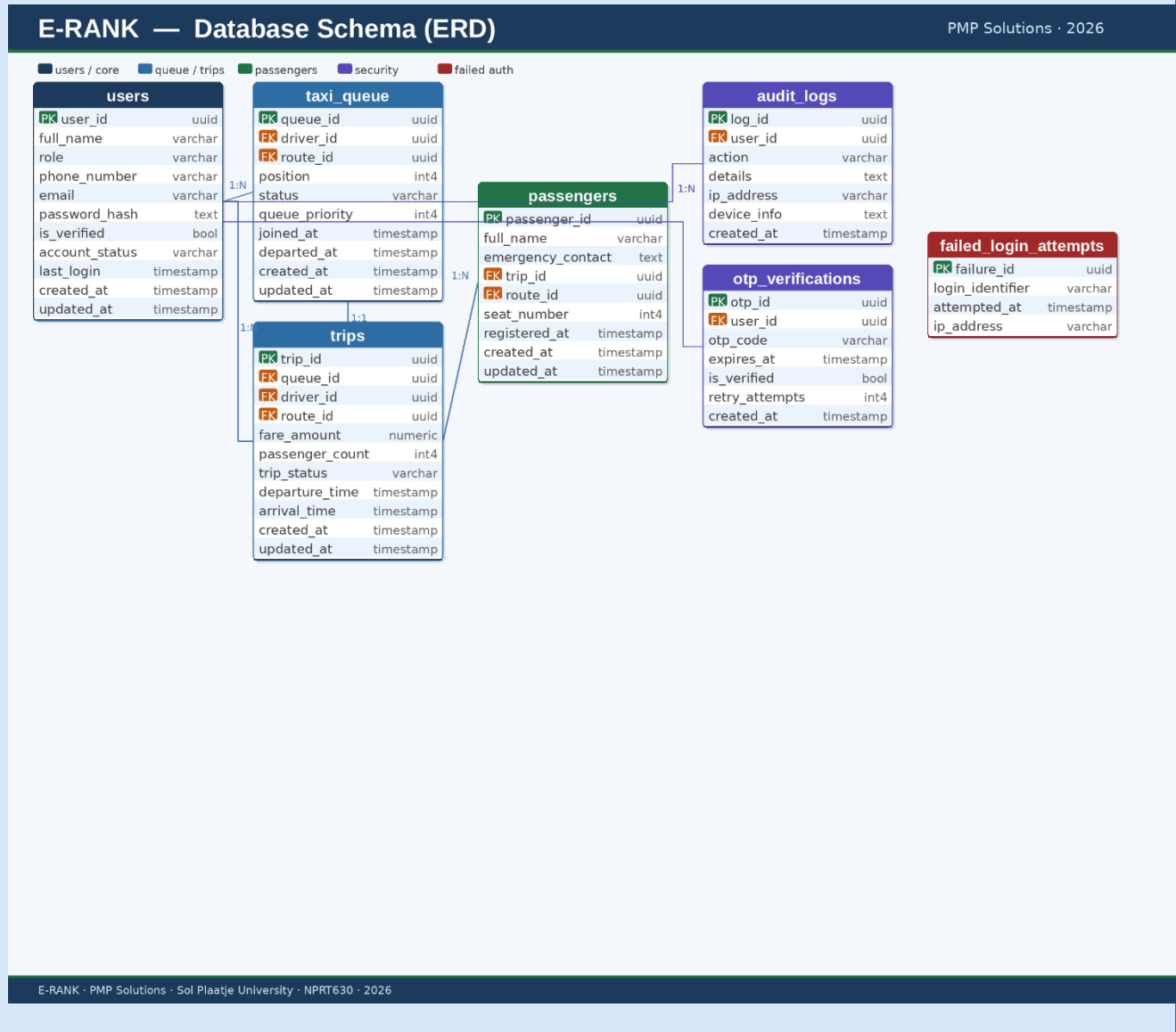
Reason: Participants reported feeling overwhelmed. Breaking the form into logical steps reduces cognitive load and improves task completion rate.

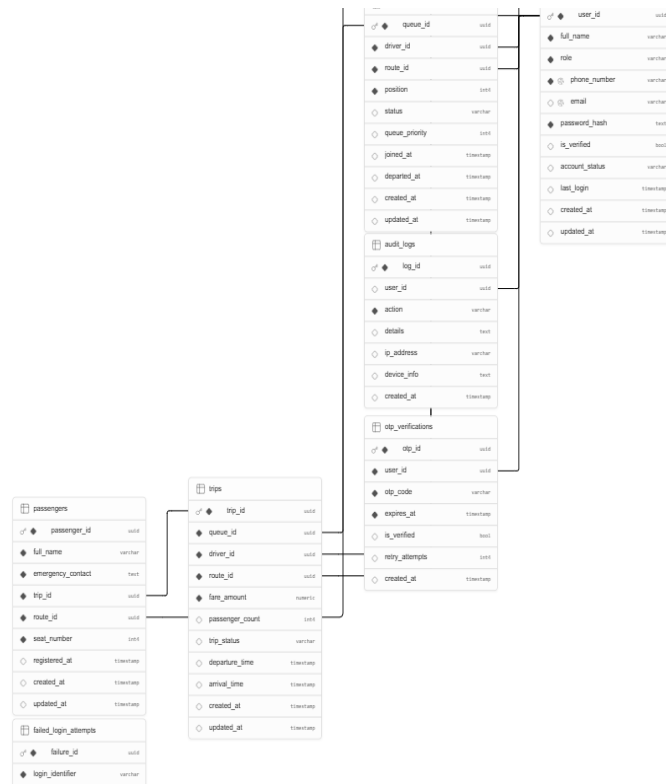
Reason: Participants struggled to find the filtering functionality. Exposing controls reduces discovery effort and user errors.

4. Entity Relationship Diagram (ERD)

The ERD below provides a visual representation of all database tables and their relationships within the E-RANK system. The diagram was derived from the Supabase PostgreSQL schema and reflects all fields, primary keys, foreign keys, and constraints implemented in the production database.

E-RANK FULL ERD – All Tables, Fields, and Relationships (Insert Supabase Schema Diagram Here)





The E-RANK database consists of the following core tables: users, passengers, trips, routes (implied), queue_entries (taxi_queue), audit_logs, and failed_login_attempts. Each table is described in detail in Section 5 below.

5. Entity Descriptions

The following section provides a field-by-field description of each table in the E-RANK database, including field name, data type, length/size constraints, and purpose.

5.1 Table: users

Stores all system users regardless of role. Authentication credentials, verification status, and account lifecycle are managed here.

users		
Field Name	Data Type	Description / Notes
user_id	UUID	Primary key. Unique identifier for each user. Auto-generated UUID.
full_name	VARCHAR	User's full legal name. Maximum 255 characters. NOT NULL.
role	VARCHAR	System role: 'driver', 'passenger', or 'admin'. Enforced by CHECK constraint.
phone_number	VARCHAR	Contact number. UNIQUE. Maximum 20 characters. NOT NULL.
email	VARCHAR	Email address. UNIQUE. Maximum 255 characters. Optional for some roles.
password_hash	TEXT	Bcrypt-hashed password. Plain text is never stored. NOT NULL.
is_verified	BOOLEAN	Indicates whether the user account has been verified. Default FALSE.
account_status	VARCHAR	Account lifecycle state: 'active', 'suspended', or 'deleted'. Default 'active'.
last_login	TIMESTAMP	Timestamp of the most recent successful login. Nullable.
created_at	TIMESTAMP	Account creation timestamp. Auto-set to current UTC time.

updated_at	TIMESTAMP	Last modification timestamp. Auto-updated on record change.
------------	-----------	---

5.2 Table: passengers

Stores passenger-specific data including trip association, seat allocation, and emergency contact information.

passengers		
Field Name	Data Type	Description / Notes
passenger_id	UUID	Primary key. Unique identifier for each passenger record.
full_name	VARCHAR	Passenger's full name. Maximum 255 characters. NOT NULL.
emergency_contact	TEXT	Emergency contact details (name and number). Free-text field.
trip_id	UUID	Foreign key referencing trips.trip_id. Links passenger to their trip.
route_id	UUID	Foreign key referencing route. Indicates route taken by passenger.
seat_number	INT4	Allocated seat number within the vehicle. Must be positive integer.
registered_at	TIMESTAMP	Timestamp when the passenger was registered for the trip.
created_at	TIMESTAMP	Record creation timestamp. Auto-set.
updated_at	TIMESTAMP	Last modification timestamp. Auto-updated.

5.3 Table: trips

Stores all trip records. Each trip is linked to a queue, a driver, and a route, and tracks the full lifecycle of a taxi journey.

trips		
Field Name	Data Type	Description / Notes
trip_id	UUID	Primary key. Unique identifier for each trip.
queue_id	UUID	Foreign key referencing taxi_queue.queue_id. Links trip to originating queue.
driver_id	UUID	Foreign key referencing users.user_id. The driver assigned to this trip.
route_id	UUID	Foreign key referencing route entity. The route this trip follows.
fare_amount	NUMERIC	Total fare charged for the trip. Must be positive. NUMERIC(10,2).
passenger_count	INT4	Number of passengers on the trip. Must be a non-negative integer.
trip_status	VARCHAR	Current status: 'pending', 'active', 'completed', 'cancelled'. NOT NULL.
departure_time	TIMESTAMP	Scheduled or actual departure time. Nullable until trip starts.
arrival_time	TIMESTAMP	Actual arrival time. Nullable until trip is completed.
created_at	TIMESTAMP	Record creation timestamp.
updated_at	TIMESTAMP	Last modification timestamp.

5.4 Table: taxi_queue

Manages the queue of drivers waiting to depart on a route. Each entry represents one driver's position in the queue at a given time.

taxi_queue		
Field Name	Data Type	Description / Notes
queue_id	UUID	Primary key. Unique identifier for each queue entry.
driver_id	UUID	Foreign key referencing users.user_id. The driver in this queue slot.
route_id	UUID	Foreign key referencing route. The route this queue entry is for.
position	INT4	Driver's numeric position in the queue. Lower number = closer to front.
status	VARCHAR	Queue status: 'waiting', 'boarding', 'departed'. NOT NULL.
queue_priority	INT4	Priority override for special cases (e.g., emergency trips). Default 0.
joined_at	TIMESTAMP	Timestamp when the driver joined the queue.
departed_at	TIMESTAMP	Timestamp when the driver departed from the queue. Nullable.
created_at	TIMESTAMP	Record creation timestamp.
updated_at	TIMESTAMP	Last modification timestamp.

5.5 Table: audit_logs

Records all significant system actions for security auditing, accountability, and monitoring of suspicious activity.

audit_logs		
Field Name	Data Type	Description / Notes
log_id	UUID	Primary key. Unique identifier for each log entry.
user_id	UUID	Foreign key referencing users.user_id. The user who triggered the action.
action	VARCHAR	Short label of the action performed (e.g., 'LOGIN', 'TRIP_CREATED'). NOT NULL.
details	TEXT	Full detail of the action in JSON or plain text format. Nullable.
ip_address	VARCHAR	IP address from which the action was performed. Maximum 45 characters (IPv6).
device_info	TEXT	Device or browser details. Nullable. Used for anomaly detection.
created_at	TIMESTAMP	Timestamp of when the action occurred. Auto-set. Immutable.

5.7 Table: failed_login_attempts

Tracks failed login attempts by login identifier to support account lockout and brute-force detection mechanisms.

failed_login_attempts		
Field Name	Data Type	Description / Notes
failure_id	UUID	Primary key. Unique identifier for each failure record.
login_identifier	VARCHAR	Email or phone number used in the failed attempt. NOT NULL.
attempted_at	TIMESTAMP	Timestamp of the failed attempt. Auto-set.

ip_address	VARCHAR	IP address of the device that made the failed attempt. Nullable.
------------	---------	--

6. Relationship Justification

This section explains the nature and justification for each relationship between entities in the E-RANK database. Relationships are implemented using foreign key constraints enforced at the PostgreSQL level.

From Entity	To Entity	Relationship	Justification
users	trips	One-to-Many	One driver (user with role='driver') can be associated with many trips over time. Each trip has exactly one assigned driver.
users	taxi_queue	One-to-Many	One driver can appear in the queue multiple times across different days or routes. Each queue entry belongs to exactly one driver.
users	audit_logs	One-to-Many	One user can generate many audit log entries (logins, actions, updates). Each log entry belongs to exactly one user.
trips	passengers	One-to-Many	One trip can carry multiple passengers. Each passenger record references exactly one trip they are travelling on.
taxi_queue	trips	One-to-One	Each queue entry, when a driver departs, corresponds to exactly one trip created. Once a queue slot is activated, a trip is created from it.
users	failed_login_attempts	One-to-Many	A user's login identifier may be used in multiple failed attempts by an attacker. This is a soft reference (by login_identifier, not FK) to track attempts even for non-existent accounts.

6.1 Relationship Type Summary

One-to-One: `taxi_queue` to `trips`. A single queue departure event creates a single trip. This ensures each trip is traceable to a specific queue departure event.

One-to-Many (most common): `users` to `trips`, `passengers`, `audit_logs`, and `taxi_queue`. This reflects the reality that a single registered user participates in many system events over their account lifetime.

Soft Reference: `failed_login_attempts` uses `login_identifier` (a string) rather than a foreign key to `user_id`. This design allows the system to log attempts for accounts that do not exist yet, which is critical for brute-force and credential stuffing detection.

7. Technology Choice and Justification

7.1 Frontend: React Native with Expo

The E-RANK mobile application was built using React Native with the Expo framework. React Native allows a single JavaScript/TypeScript codebase to compile and run natively on both Android and iOS devices. Expo provides additional tooling including over-the-air updates, easy device testing, and simplified build pipelines. This satisfies the assignment requirement for Java/code-based implementation as React Native compiles to native Java (Android) and Swift (iOS) code.

- Cross-platform compatibility: One codebase, two platforms.
- Expo Router: File-based navigation, similar in pattern to Next.js.
- React Native Paper: UI component library for consistent Material Design styling.
- Fast development cycle with Expo Go for live device testing.

7.2 Backend Runtime: Node.js

Node.js serves as the backend runtime environment for the E-RANK API layer. It handles incoming HTTP requests, processes business logic, and communicates with the Supabase PostgreSQL database. Node.js was chosen for its non-blocking event-driven architecture, which is well-suited to handling concurrent requests from multiple mobile clients simultaneously. The Express.js framework was used to structure API routes, middleware, and error handling cleanly.

- Non-blocking I/O: Efficiently handles multiple simultaneous user requests.
- Express.js: Lightweight and flexible REST API framework.
- JSON-native: Natural integration with React Native frontend and PostgreSQL data.

- Environment variable management via dotenv for secure credential handling.

7.3 Database: PostgreSQL via Supabase

The E-RANK backend database was built on PostgreSQL, hosted on Supabase. PostgreSQL was selected for its enterprise-grade reliability, relational integrity enforcement, and advanced security features. Supabase was chosen as the cloud hosting platform because it provides managed PostgreSQL hosting with built-in authentication support, REST and real-time API generation, and Row Level Security (RLS) — removing the limitation of local-only database access.

- **UUID Primary Keys:** Improve security, uniqueness, and distributed scalability.
- **Row Level Security (RLS):** Ensures users can only access data they are authorised to view.
- **Indexing:** Applied to foreign keys, email, phone number, timestamps, and status columns for query performance.
- **CHECK Constraints:** Enforce valid roles, positive fare amounts, and valid status values at the database level.
- **Security Tables:** Dedicated tables for audit logs and failed login attempts support authentication monitoring and attack detection.
- **Cloud-Based Architecture:** Allows multi-device access, remote connectivity, and mobile integration.

7.4 Summary Technology Stack

Layer	Technology	Purpose
Mobile Frontend	React Native + Expo	Cross-platform iOS/Android application
UI Components	React Native Paper	Material Design component library
Backend Runtime	Node.js + Express.js	REST API, business logic, request handling
Database Engine	PostgreSQL	Relational data storage with integrity constraints
Cloud Platform	Supabase	Managed PostgreSQL, auth, RLS, real-time APIs

Authentication	Password Hashing (Bcrypt)	Secure password storage; plain-text passwords never stored
Version Control	Git + GitHub	Source code management and collaboration

8. Unit Test Cases

The following test cases were defined and executed to validate the core modules of the E-RANK system. Testing was conducted as part of Phase 3 quality assurance. Tests were performed by the development team in a controlled environment. Usability tests were performed in Moroka Seminar Room 112, Sol Plaatje University, with real users. Unit tests were run programmatically against the Node.js API and Supabase database.

Test Document Prepared by: Siyabonga José Ndzobondzobo (202441850)

8.1 Authentication & User Registration Tests

Step	Test ID	Test Case Description (3 Marks)	Input / Action (3 Marks)	Expected Output (3 Marks)	Pass/Fail Criteria (3 Marks)
1	TC-A001	Access Registration screen from app launch	User opens app and taps 'Register'	Registration form loads with all required fields visible	Pass
2	TC-A002	Enter valid registration details	User enters full name, phone, email, valid password, selects role 'Passenger'	All fields accepted; account created; user redirected to Login	Pass
3	TC-A003	Enter duplicate email during registration	User enters email already in database	System returns error: 'This email is already registered'	Pass
7	TC-A007	Login with correct credentials	User enters registered email and correct password	User is authenticated and redirected to role dashboard	Pass
8	TC-A008	Login with incorrect password	User enters correct email but wrong password	Error: 'Invalid credentials';	Pass

				failed_login_attempts record created	
--	--	--	--	---	--

8.2 Queue and Trip Management Tests

Step	Test ID	Test Case Description (3 Marks)	Input / Action (3 Marks)	Expected Output (3 Marks)	Pass/Fail Criteria (3 Marks)
1	TC-Q001	Driver joins the queue for a route	Driver selects route and taps 'Join Queue'	Driver added to taxi_queue; position assigned based on current queue length	Pass
2	TC-Q002	Queue position displayed correctly	Driver views dashboard after joining queue	Dashboard shows correct numeric position and status 'waiting'	Pass
3	TC-Q003	Driver departs from queue	Driver taps 'Depart' when vehicle is full	Queue entry status updated to 'departed'; departed_at timestamp recorded	Pass
4	TC-Q004	Trip record created on departure	System processes driver departure	New trip record created in trips table with status 'active'	Pass
5	TC-Q005	Passenger added to trip	Marshal or passenger submits passenger registration form	Passenger record created and linked to correct trip_id and route_id	Pass
6	TC-Q006	Seat number conflict	Two passengers assigned same seat on same trip	System returns error: 'Seat already taken on this trip'	Pass
7	TC-Q007	Trip completed	Driver marks trip as complete on arrival	Trip status updated to 'completed'; arrival_time recorded	Pass

8.3 Admin and Audit Tests

Step	Test ID	Test Case Description (3 Marks)	Input / Action (3 Marks)	Expected Output (3 Marks)	Pass/Fail Criteria (3 Marks)
1	TC-AD001	Admin views all users	Admin logs in and navigates to User Management	All registered users displayed with name, role, and status	Pass
2	TC-AD002	Admin deactivates a user account	Admin selects a user and taps 'Suspend'	User account_status set to 'suspended'; user cannot log in	Pass
3	TC-AD003	Audit log generated on login	Any user successfully logs in	New record created in audit_logs with action='LOGIN' and correct user_id	Pass
4	TC-AD004	Audit log generated on failed login	Any user fails login attempt	New record in failed_login_attempts and audit_logs with action='FAILED_LOGIN'	Pass
5	TC-AD005	Admin generates trip report	Admin selects date range and taps 'Generate Report'	Report displays total trips, passengers, and fare for selected period	Pass
6	TC-AD006	RLS prevents cross-user data access	Driver attempts to query another driver's records via API	Request rejected with 403 Forbidden; no data returned	Pass

— End of Phase 3 Report —

E-RANK | NPRT630 Group Assignment 2026 | Sol Plaatje University